# ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

**Please do not change the style of this document.**

**Question numbers 1–10 follow the same order as the notebook** (top to bottom).

---

**Student Information**

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.


- **English Name: Tsung-Min (Sandra), Chang**
- **Chinese Name: 張宗旻**
- **Email: chendlar.coffee@gmail.com**
- **Student ID: 115062518**
- **University: National Tsing Hua University (NTHU)**


---

## Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

**How to answer each question**

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

**Sources, honesty, and AI use**

**Acceptable vs. Unacceptable AI Usage:** Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

---

## Questions

### Question 1: Data Filtering & Domain Relevance

**Notebook:** Section 3.2 — Question 1 (take home)

**Task:** Filter the dataframe **X** for the category **sci.med** and check its length.

**Questions:**

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

### Answer:

(1) There are 594 documents in the sci.med category.

(2) Splitting by category first is important; otherwise we end up learning very general patterns instead of patterns that represent each category. Per-category mining keeps support values meaningful. For example, suppose {patient, doctor} appears in 60 documents, all in sci.med, and the minimum support is 5%. Mined within sci.med alone, its support is 60/594 ≈ 10%, so it passes the threshold. Mined in the mixed pile of all 2,257 documents, its support drops to 60/2,257 ≈ 2.7%, so it fails and is discarded. Meanwhile, generic combinations like {know, think} appear across all topics and pass easily in the mixed pile, so we end up with patterns that tell us little about any single category.

**AI used for this answer:** Yes

**If yes, how:** double check concept, grammar polishing

**External references (links):** https://en.wikipedia.org/wiki/Association_rule_learning

---

**Question 2: Handling Missing Data**

**Notebook:** Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

**Task:** In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

**Question:** If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

**Answer:**

Simply dropping these rows is flawed because 40% of the data would be thrown away, and with less data, fewer patterns reach the support threshold, so the patterns we find become weaker. It can also bias the results: if the missing labels are not random (for example, one category is unlabeled more often), the remaining 60% no longer reflects the true category distribution. A better alternative is to predict the missing labels from the known ones: train a classifier on the 60% of documents that have labels, then use it to fill in the remaining 40%, which is a semi-supervised learning approach. I prefer this over labeling them "Unknown", because treating "Unknown" as a class would teach the model to predict "Unknown" for new data, which gives no useful information.

**AI used for this answer:** Yes

**If yes, how:** double check concept, grammar polishing

**External references (links):** https://scikit-learn.org/stable/modules/semi_supervised.html and https://pandas.pydata.org/docs/user_guide/missing_data.html

---

**Question 3: Data Sampling Risks**

**Notebook:** Section 5.1 — Question 3 (take home)

**Task:** In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

**Questions:**

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

**Answer:**

(1) No. The proportions are close but not exact: comp.graphics rose from 25.9% to 27.8%, while soc.religion.christian dropped from 26.5% to 24.8%. Because the top three categories are nearly tied in the original data, even these small random shifts changed their ranking (original: soc.religion.christian, sci.med, comp.graphics, alt.atheism; sample: comp.graphics, sci.med, soc.religion.christian, alt.atheism). I also noticed that the notebook's bar chart pairs the two value_counts() series by position rather than by category name, so after the ranking changed, the sampled bars for comp.graphics and soc.religion.christian were plotted under each other's labels.

(2) If the dataset is highly imbalanced, a rare category may be under-represented or even missed entirely by simple random sampling. A model trained on that sample would then rarely or never see examples of the rare class and could not learn it. Stratified sampling avoids this by sampling the same proportion from each category.

**AI used for this answer:** Yes

**If yes, how:** double check concept, debugging the chart, grammar polishing

**External references (links):**
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sample.html and
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

---

**Question 4: Text Vectorization and Matrix Sparsity**

**Notebook:** Section 5.3 — Question 4 (take home)

**Task:** In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

**Questions:**

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

**Answer:**

(1) The number of features increases, from 35,482 to 35,788 columns (+306). Without stop-word removal, the common English words become part of the vocabulary, so each one adds a column. The increase is small (about 0.9%) because the stop-word list only has around 300 words.

(2) Sparsity decreases, from 99.68% to 99.55%. Stopwords are common and appear in many documents, so each stop-word column adds many non-zero entries to the matrix: the non-zero count jumped from 258,503 to 365,886 (+41%), even though only 306 columns were added. The overall sparsity drops only slightly because the matrix is still dominated by more than 35,000 rare words that appear in only a few documents each.

**AI used for this answer:** Yes

**If yes, how:** double check concept, code for computing sparsity, grammar polishing

**External references (links):** https://scikit-learn.org/stable/modules/feature_extraction.html#stop-words

---

**Question 5: Feature Selection — Variance Thresholding**

**Notebook:** Section 5.3.1 — Question 5 (take home)

**Task:** In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

**Questions:**

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

**Answer:**

(1) The vocabulary shrinks quickly as the threshold increases: 35,482 terms are kept at 0.0 (every term appears at least once, so its variance is above zero), 5,089 at 0.01, and only 1,491 at 0.05. Only terms whose counts vary enough across documents survive; the rest are cut off.

(2) Terms with very low document frequency have near-zero variance, meaning words that appear in only one or a few documents. Their columns are 0 in almost every document, so the values barely change and we see them as almost constant. The lowest-variance terms removed were typos ("haveing", "hawe"), rare names ("hatfield", "haussler") and encoding junk ("íålittin"), each with variance 0.000443, which is what a word appearing once in one of the 2,257 documents gives.

(3) A feature that is almost constant across documents cannot help distinguishing one document or category from another, and it is too rare to form any pattern to learn from. Removing it reduces dimensionality and noise while losing very little information.

| Threshold | Number of terms kept (keep == True) |
|---|---|
| **0.0** | 35482 |
| **0.01** | 5089 |
| **0.05** | 1491 |

**AI used for this answer:** Yes

**If yes, how:** double check concept, debugging the notebook error, grammar polishing

**External references (links):** https://scikit-learn.org/stable/modules/feature_selection.html#removing-features-with-low-variance

---

**Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)**

**Notebook:** Section 5.3.2 — Question 6 (take home)

**Task:** Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

**Questions:**

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

**Answer:**

(1) They're not stopwords, because stopwords were already removed before this step (stop_words='english'). The Spearman list is mostly real graphics words, like graphics, files, image, 3d, program and windows. It also picks up "people" and "god" as negative signals, which basically means "this post is probably about religion, not graphics." The Pearson list is messier. It has a few graphics words (vga, video, card), but also words like "thanks", "hi" and "looking", probably because a lot of graphics posts are people asking for help, plus email header words like "writes" and "host". Interestingly, the word "graphics" itself only shows up at #12 in Pearson.

(2) Pearson looks at the raw counts and checks whether they follow a straight-line (linear) pattern, so a few posts with really big counts can throw it off. Spearman first turns the counts into rankings, then checks whether graphics posts consistently rank higher than other posts (a monotonic pattern). It doesn't care how big the gaps are, so it's much less affected by outliers.

(3) This happens when a word shows up regularly in the topic's posts, but a few posts use it a crazy number of times. Those few extreme posts mess up Pearson, but Spearman only cares that the topic's posts rank higher, so it still gives a high score. My results show exactly this with the word "graphics": Pearson gave 0.18, but Spearman gave 0.52.

**AI used for this answer:** Yes

**If yes, how:** double check concept, concept explanation with a toy example, grammar polishing

**External references (links):**
https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient and
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

**Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics**

**Notebook:** Section 5.4.2 — Question 7 (take home)

**Task:** In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

**Questions:**

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

**Answer (1) — Runtime and pattern count when lowering minSup:**

In plain words, this is like lowering the bar from "you get in if you show up at least 9 times" to "you get in if you show up at least 3 times." Way more patterns make it in, and it takes more time to check everyone. In my run on alt_atheism, the number of patterns jumped from 710 to 9,547 (about 13x), and the runtime went from 0.046s to 0.886s (about 19x slower).

**Answer (2) — Top-K k vs MaxFPGrowth minSup:**

minSup is like a bar: anyone above it gets in, no matter how many people that ends up being. k works the other way around: we only let in a fixed number of people (the top k), and the bar adjusts itself to fit. You can see this in my results: with k=100 the lowest support that made the list was 21, and with k=500 it dropped to 16.

**Answer (3) — Maximal vs standard frequent patterns:**

Because we only keep the biggest set, the "whole family." Any smaller set that's already inside a bigger frequent set doesn't get listed separately, since it's already represented. For example, the 6-word pattern "beaverton bronx manhattan queens blew sank" has 63 smaller combinations inside it that are all frequent too, but maximal mining only lists the one big set.

**AI used for this answer:** Yes

**If yes, how:** double check concept, grammar polishing

**External references (links):** https://github.com/UdayLab/PAMI and https://en.wikipedia.org/wiki/Association_rule_learning

---

**Question 8: Dimensionality Reduction — Motivation**

**Notebook:** Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

**Task:** After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

**Questions:**

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

**Answer:**

(1) Each document is a row with 35,482 numbers, one for each word, so every document is a point in a 35,482-dimensional space. A scatter plot only has 2 axes (or 3 in 3D), and we can't see more than 3 dimensions. We also can't just pick 2 words as the axes, because the matrix is 99.68% zeros: most documents would have 0 for both words and pile up at the same spot, so the plot wouldn't show anything useful.

(2) We first need to apply dimensionality reduction algorithms. They squeeze the 35,482 dimensions down to 2 or 3 while trying to keep similar documents close together, kind of like a shadow that flattens a 3D object but keeps its shape. In the notebook, the helper file visualization_helpers.py uses PCA, t-SNE and UMAP to bring the data down to 2D so we can show it on a scatter plot.

**AI used for this answer:** Yes

**If yes, how:** double check concept, grammar polishing

**External references (links):** https://en.wikipedia.org/wiki/Dimensionality_reduction and https://scikit-learn.org/stable/modules/decomposition.html#pca

---

**Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)**

**Notebook:** Section 5.5 — Question 9 (take home) (after 2D/3D plots)

**Task:** In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

**Questions:**

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

**Answer:**

(1) t-SNE and UMAP show separated cluster regions, while PCA shows heavy overlap: almost all posts get squashed into one blob near the center, with just a few outliers far away. In UMAP, for example, christian posts sit at the top, atheism on the left, graphics at the bottom and sci.med on the right, with atheism and christian right next to each other since both are about religion. But honestly, the separation is only partial: the middle parts of t-SNE and UMAP are still heavily overlapped.

(2) PCA is like a flat shadow. It finds the 2 straight directions with the most spread, and here those directions are mostly driven by a few very long posts. Text categories are tangled in curvy ways, and a flat projection can't untangle them. t-SNE and UMAP instead focus on keeping close neighbors close, meaning posts that use similar words stay near each other, so the topic groups show up much better.

(3) t-SNE only tries to keep close neighbors close. It doesn't try to keep the distances between groups accurate, so the gap between two clusters, and even how big each cluster looks, don't really mean anything. The axes and directions are arbitrary too, and the picture can change a lot with a different perplexity or random seed. What we can trust is which posts end up grouped together, not how far apart the groups are.

**AI used for this answer:** Yes

**If yes, how:** double check concept, grammar polishing

**External references (links):** https://distill.pub/2016/misread-tsne/ and https://umap-learn.readthedocs.io/en/latest/how_umap_works.html

---

**Question 10: One-Hot Binarization**

**Notebook:** Section 5.6 — Question 10 (take home)

**Task:** In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

**Questions:**

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

**Answer:**

(1) 4 binary columns, one for each category. For example, a comp.graphics post (category 1) becomes [0, 1, 0, 0], and a soc.religion.christian post (category 3) becomes [0, 0, 0, 1].

(2) The algorithm assumes the categories have an order and a distance, like 0 < 1 < 2 < 3, when they're really just name tags and should be treated like text labels ("0", "1", "2", "3"). This leads to weird conclusions. For example, KNN would think alt.atheism (0) is only 1 step away from comp.graphics (1) but 3 steps away from soc.religion.christian (3), as if atheism were more similar to computer graphics than to christianity, which makes no sense. Even the "average" of atheism (0) and sci.med (2) would come out as comp.graphics (1).

(3) Each category gets its own switch, and only one switch is on at a time. No category is bigger than another, and every pair of categories is the same distance apart, so the algorithm can't invent a fake order or fake similarity between them.

**AI used for this answer:** Yes

**If yes, how:** double check concept, debugging the notebook error, grammar polishing

**External references (links):**

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html and https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features

---